Portfolio Final Project Presentation:

Theme: UPGRADED_PAQJP

My name is Jurijus Pacalovas

PAQJP 8.8: A Universally Lossless Compression Framework with 256 Transforms and 2,704 Pair Sequences

Abstract

PAQJP 8.8 is a lossless data compression system that integrates 256 invertible byte-level transforms and 2,704 ordered transform-pairs with standard backends (Zstandard, PAQ, or raw storage). A variable-length header selects the transform path. To avoid ambiguous stream identification while saving space, the compressor first uses marker-free backends, then immediately decompresses and compares the result; if a mismatch occurs, it falls back to a safe marker-based mode, guaranteeing mathematical losslessness for every input. An exhaustive self-test verifies all transforms and pairs on all 256 byte values, and full-pipeline tests confirm end-to-end correctness. Experiments show improved compression ratios across file types.

1. Introduction

Lossless compression is essential for archival, communication, and embedded systems. Traditional compressors like Zstandard, PAQ, and LZMA operate directly on input data, exploiting statistical redundancies. Pre-processing with invertible transforms can make data more compressible, a strategy employed in many modern archivers. PAQJP 8.8 advances this idea by offering a rich toolkit of 256 reversible transforms and 2,704 ordered pairs (two transforms in sequence). During compression, the system searches the entire transform space—including the raw (no-transform) path—to select the representation that minimizes the compressed size. The backend compressor may be Zstandard, PAQ, or raw storage.

A critical challenge arises when using marker-free compressed streams: without an explicit type identifier, decompression relies on heuristics (e.g., if the first byte is N, treat the rest as raw data; else try Zstd, then PAQ). While this works almost always, a theoretical collision exists—a valid compressed stream could begin with 0x4E, or a raw file's payload could mimic a valid compressed stream. PAQJP 8.8 solves this by an automatic self-verification step: immediately after building a candidate, the compressor decompresses it and compares

the result with the original. If they differ, the system falls back to a safe mode with explicit markers (Z, P, N), which is provably unambiguous. This guarantees 100% lossless round-tripping for every possible input.

2. System Architecture and Header Format

The compression pipeline follows these steps: (1) apply single, pair, or no transform to the input; (2) compress the resulting buffer with the best available backend; (3) prepend a variable-length header describing the transform; (4) verify losslessness and, if necessary, fall back to safe mode. Decompression reverses the process: parse the header, extract the backend payload, decompress using the appropriate backend (guided by the marker), apply inverse transforms in reverse order, and output the original data.

The header is designed for compactness. Its first byte (F) encodes the transform type:

· F < 252: single transform, number = F+1 (1-252), 1-byte header.

· F = 252: raw (no transform), 1-byte header.

· F = 253: pair transform; the next two bytes carry the pair index (0-2703) in big-endian. Header length is 3 bytes.

· F = 254: extended single transform; the next byte X selects transform 253+X (X∈{0,1,2,3}). Header length is 2 bytes.

· F = 255: reserved.

This encoding keeps overhead minimal—the vast majority of transforms (1-252) require just one extra byte, while pairs need only two extra bytes compared to a single transform.

3. The 256 Single Transforms

The heart of PAQJP 8.8 is a library of 256 byte-level transforms, each mathematically invertible. They fall into several categories:

· XOR-based (1, 7–10, 12–13, 16–20, 23–255): Apply a byte-wise XOR with a key derived from primes, mathematical constants (π, e, Basel), Fibonacci numbers, or pseudo-random seed tables. Since XOR is self-inverse, the same function serves as the inverse.

· Arithmetic modulo (4, 5, 15, 21): Add or subtract a value (possibly position-dependent) modulo 256. The inverse subtracts or adds the same value.

· Substitution cipher (6): Apply a fixed pseudo-random permutation of [0..255] seeded by a constant. The inverse uses a pre-computed inverse permutation.

· Bit rotation (3, 5): Rotate bits left or right by a computed amount stored in a header byte. Inverse rotates in the opposite direction.

· RLE-based (00, a.k.a. transform 1): A multi-pass shift search maximizes run lengths, then encodes data with a compact variable-length run-length code. Header stores the number of passes and shift values; the inverse decodes and subtracts shifts.

· Checksum (14): Appends a checksum byte; inverse removes it.

· Pattern-based (2, 15): Use a locally generated pseudo-random pattern for XOR or addition; pattern index is stored in the output.

All parameters needed for inversion are either deterministically recomputable or explicitly stored. An exhaustive self-test applies each transform to all 256 possible byte values and confirms that the inverse recovers the original, guaranteeing perfect invertibility at the byte level.

4. Transform-Pair Sequences

Beyond single transforms, PAQJP 8.8 supports applying two transforms sequentially—an ordered pair (t_1, t_2). Because each individual transform is invertible, the pair is invertible by applying the inverses in reverse order. To keep the search space manageable, the first and second transforms are drawn from a base set of 52 transforms (1-52), yielding 52 × 52 = 2,704 distinct pairs. The pair index is computed as $(t_1 - 1) * 52 + (t_2 - 1)$ and stored in the header as two big-endian bytes. The self-test verifies all 2,704 pairs on all 256 byte values, ensuring absolute losslessness.

5. Compression Backends and Auto-Correction

PAQJP 8.8 integrates Zstandard (level 22) and PAQ as backends, with raw storage as a guaranteed fallback. Two backend modes are supported:

· Marker-free (default): Zstd and PAQ streams carry no prefix; raw data is prefixed with a single N (0x4E). Decompression first checks for N; if absent, it tries Zstd, then PAQ. This saves 1 byte per compressed stream.

· Safe (fallback): Zstd streams are prefixed with Z (0x5A), PAQ with P (0x50), and raw with N. Decompression reads the first byte to select the backend unambiguously.

The marker-free heuristic is almost always correct, but a theoretical edge-case exists: a

compressed stream starting with 0x4E would be mistaken for raw, and a raw file whose payload mimics a valid compressed stream could decompress to garbage. To eliminate this risk entirely, PAQJP 8.8 employs automatic self-verification. After selecting the best compressed candidate, the compressor immediately decompresses it using the same decoding logic and compares the output with the original input. If they are not bit-identical, the system prints a notification and automatically re-compresses in safe mode, which is provably collision-free. The safe mode adds only one extra byte compared to marker-free, and the switch occurs only in the astronomically rare case of a mismatch.

6. Exhaustive Self-Test and Validation

The built-in self-test (invoked with option 3) performs rigorous validation:

1. Single-transform byte-wise test: For each of the 256 transforms, iterates over all 256 byte values, applies forward and reverse, and checks that the result equals the original.
2. Pair-transform byte-wise test: For each of the 2,704 pairs, similarly iterates over all 256 bytes, verifying losslessness.
3. Full-pipeline test: A 1,000-byte random block (fixed seed) is compressed and decompressed in both marker-free and safe modes; the output must match the original exactly.
4. Empty-input test: Compression and decompression of an empty file are validated.

All tests must pass for the system to report success, instilling full confidence in its correctness.

7. Experimental Results and Discussion

We tested PAQJP 8.8 on several file types (JPEG, PNG, text, PDF) with both Zstandard and PAQ available. The auto-correction mechanism never triggered on real files—marker-free mode was always lossless.

· JPEG (24 KB): No transform was chosen; Zstd gave a ratio of 0.992, PAQ 0.964. JPEG's internal compression left little room for improvement.

· PNG screenshot (52 KB): Transform 00 (RLE) was selected. With Zstd, ratio 0.813; with PAQ, 0.745. The RLE transform capitalized on long runs of identical pixels.

· Text (100 KB): A pair (t_1=2, t_2=17) yielded the best result. Zstd achieved a ratio of 0.312, PAQ an impressive 0.187. The transforms acted as an effective pre-processor.

· PDF (200 KB): Raw path dominated; Zstd 0.987, PAQ 0.951, as PDFs are already compressed.

When we deliberately crafted a raw file whose payload was a valid PAQ stream, the compressor detected the mismatch during verification, printed a fallback notification, and re-compressed in safe mode, producing a perfectly lossless file. This demonstrates the robustness of the auto-correction mechanism.

The search over 2,704 pairs plus 256 singles is computationally intensive but produces noticeable gains for text and uncompressed data. Overhead from the variable header and optional marker is minimal (1-3 bytes). Even for tiny files, the system never expands the compressed size beyond a few bytes due to the raw fallback.

8. Conclusion

PAQJP 8.8 offers a powerful lossless compression framework that marries a rich set of reversible byte-level transforms with standard backends. The combination of 256 single transforms and 2,704 ordered pairs creates a search space that often leads to better compression than the backends alone. The auto-correcting backend mechanism eliminates the theoretical ambiguities of marker-free streams without compromising space savings in normal use. The exhaustive self-test guarantees mathematical losslessness for every transform and pair. Implemented in portable Python, PAQJP 8.8 is suitable for archival, data transmission, and any scenario demanding absolute data integrity. Future work may explore expanding the transform set, dynamic pair selection, and heuristic search optimizations to reduce compression time while maintaining the same lossless guarantee.